Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Timothy May	mayt@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I want to make a blackjack web game where players can play against a computer dealer using pretend money. Everyone will start with $100, and the goal is to make choices during each hand while keeping track of their balance, debt, and game results.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

Free online blackjack practice games already let people play against a computer dealer using virtual chips. I want to use that basic idea but add my own bank page, a debt system, game history, and statistics. I will not use real money.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

My Drafter website will have a home page, blackjack table, bank, statistics, game history, and rules page. Players will start with $100 (fake money) and choose their bet before a round. They can hit, stand, fold, or double down (when their first two cards total below 11),

and they have enough money to double down. Players may borrow only when their balance is $0, and they can repay what they owe when they have money. The site will show previous hands and how many games they have won or lost.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

My program will use Drafter routes, inputs, State data, lists, functions, and if statements to play blackjack with virtual money and keep track of results, balance, and debt.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

Players will type a bet amount, a borrowing amount, and a debt repayment amount into three separate TextBoxes. They will also use buttons to hit, stand, fold, double down, move between pages, and start a new hand. Everyone starts with $100, so the starting amount is not an input.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

The program will randomly deal card values, it will calculate blackjack hand totals (including aces), and compare the player's hand with the dealer's. If statements will check if the card value is over 21, the dealer hits below 17, legal bets, double-down limits, borrowing only at $0, and valid repayments. Routes will update the balance, debt, bet, cards, win/loss counts, and a list of previous rounds. A loop will read the game records to display the history.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The website will show the player and dealer cards, current hand totals, the current bet, available moves, and the result of each round. Other pages will show the player’s pretend

balance and debt, total wins and losses, previous games, and the game rules. Invalid actions will show clear messages.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

The important things are the player's game state, the cards, the bet, the dealer's hand, the loan balance, and the records of finished rounds. I will represent a finished round with a GameRecord dataclass and store a list of those records inside a State dataclass.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Details I need for State: balance, debt, wins, losses, history (a list of GameRecord objects), current bet, player cards, and dealer cards. Each GameRecord will store the bet, result, player total, and dealer total. I will ignore physical card size, card condition, real banking details, and real payments because this is just a pretend-money game.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

I think the most difficult parts will be getting the blackjack rules right, handling aces, and making sure the player cannot cheat the bank or change their money incorrectly. I also need to keep information updated across different Drafter routes and build at least five working pages. Testing the randomized cards and checking all the special cases will take extra work.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will build and test the game a little at a time over the course of a couple of weeks. If I use an LLM, I would ask it to explain the code to me, then check the answer against our class examples and the Drafter documentation, and make sure that I understand it before using it. I will write Bakery tests for functions and routes when possible and explain in comments when random behavior makes a route hard to test.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

I want to make a working blackjack game that looks good and is easy to understand. My goal is to learn more about Drafter routes, dataclasses, lists, loops, functions, and testing. I also want to meet every final-project requirement and have a working website that I can deploy and show in my video.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to make a Drafter blackjack game where everyone starts with $100 in pretend money. Players can hit, stand, fold, and double down below 11 if they have enough money. They can borrow only when they have $0 and repay debt when they have money. How can I turn this into a project with at least five pages, seven routes, three inputs, two dataclasses, and a game history without making it too hard?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The strongest suggestions were to create six pages (home, game, bank, stats, history, and rules), use separate routes for actions, and store finished hands in a list of GameRecord objects. The game history and stats pages will make the project more useful and help meet the programming requirements. Existing free blackjack practice games are examples of virtual-chip gameplay, but my version will also include debt tracking.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

A feature like splitting hands or adding insurance would make the game much harder, especially because I would need to keep track of multiple bets and hands at once. I would rather focus on hit, stand, surrender, and double down first. A simpler alternative is to have one player hand, one dealer hand, and a history page that saves each result.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

I originally wanted a blackjack game with money and borrowing. Now the idea is more organized because I have six planned pages and specific things to store in State. I will keep the game to one player hand at a time, use pretend money, and include game history and statistics so I can meet the class requirements without adding every casino rule.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

I will use the course notes, class examples, Drafter documentation, Bakery documentation and tests, my professor or TA, and ChatGPT when I need help understanding a problem. I will also look back at the project requirements after each major change.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

First, I will read the error and find the route or function causing it. Second, I will check the State fields and the input values. Third, I will test smaller helper functions with Bakery and

check the routes that do not use randomization. Fourth, I will compare my Drafter functions with class examples and documentation. Finally, I will run and deploy the website again, click through every page, and make sure the issue is gone.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

For my final project, I want to make a blackjack game using Python and Drafter. The player will start with $100 in fake money and be able to place bets, hit, stand, surrender, and double down. I also want to make it so players can borrow money when they have $0 and pay back their debt whenever they have enough money. What are some problems I might run into, and what features should I add to make the game better?

Some of the biggest challenges will be making sure the cards work correctly, calculating the player's and dealer's totals, and keeping track of money between rounds. You will also need to make sure players cannot bet more money than they have or borrow money unless their balance is $0.

Some features you could add are a game history page, player statistics, and a bank page where players can borrow money and pay back debt. You could also add different difficulty levels, computer opponents, or animations for dealing cards. Websites like Blackjack Apprenticeship and 247 Blackjack have online blackjack games that could give you ideas for the layout.

I like the idea of adding player statistics and game history, but I think adding different difficulty levels, computer opponents, and animations might be too complicated since this is only my first computer science class. What would be a simpler way to make the game interesting while still meeting all the requirements for the project?

I would recommend keeping the game to one player against a computer dealer that follows normal blackjack rules. Instead of different difficulty levels and animations, you could add a

statistics page that shows how many games the player has won or lost and a game history page that shows their previous bets and results.

You could also have a separate bank page for borrowing and repaying fake money. These features would make the game more interesting without making the code too complicated. They would also help you meet the requirements for multiple pages, routes, dataclasses, and loops.

That sounds better. I think I'm going to keep the game simple with one player against the dealer. I still want the player to start with $100 and be able to borrow money only when they have $0. I also want the game to keep track of wins, losses, and previous rounds. Do you think this would be enough for my final project?

Yes, this would be a good project for an introductory Python course. You could organize the website into six pages: Home, Blackjack Table, Bank, Statistics, Game History, and Rules.

You could also use a `State` dataclass to track the player's balance, debt, wins, losses, cards, and current bet. A second dataclass called `GameRecord` could store the results of previous rounds in a list.

This would give you enough features to meet many of the course requirements while keeping the game manageable. You would still need to make sure you have at least seven routes, three input fields, meaningful if statements, and tests for your functions.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.